59. KubeRay for Heavy Computation: Examples, Local vs Production

Builds on **#51 (AWS)**, **#53 (Kubernetes)**, **#55 (KubeRay basics)**, **#56 (Node threads)** and **#57 (BullMQ + KEDA)**.

> **In one line:** yes, **KubeRay is made for heavy calculations**: work too big or too slow for one computer. It runs **Ray** (Python) on Kubernetes and splits the work across many pods and machines. You test it **small on your laptop**, then run it **big in the cloud** with the **same code**.

PART 1: Why KubeRay for heavy work

1. The kitchen example

> **1 chef cooking 1,000 plates = 10 hours. 100 chefs in a big kitchen = about 6 minutes.**
>
> **Ray** = the **head chef**, who splits the orders between the chefs.
> **Ray workers** = the **chefs**, who cook at the same time.
> **KubeRay** = the **restaurant manager**, who hires extra chefs (pods) when it's busy and sends them home when it's done.
> **Kubernetes** = the **building** (kitchens = machines/nodes).

Word	Meaning
Ray	A Python library that runs functions on **many machines** as if they were one big computer
KubeRay	A Kubernetes **operator** that creates, scales and deletes Ray clusters for you
RayCluster	A long-running cluster: 1 head pod + many worker pods
RayJob	Creates a cluster, runs **one script**, then deletes the cluster (you pay only while it runs)
RayService	Serves a model as an **API**, autoscaled with traffic
`@ray.remote`	A decorator meaning "this function can run on any worker pod"

PART 2: Heavy computation examples

2. Example 1: process 10,000 student photos

The school uploads **10,000 photos**. Each one must be **resized** and have **faces detected**. One photo takes about **2 seconds** of CPU.

Setup	Calculation	Time
1 machine, 1 core	10,000 x 2 s	**about 5.5 hours**
KubeRay: 50 pods x 4 CPU = 200 at once	10,000 / 200 x 2 s	**about 2 minutes**

```
# process_photos.py
import ray
ray.init()   # connects to the Ray cluster that KubeRay created

@ray.remote                       # "this function can run on any pod"
def process_photo(path):
    img = load(path)
    img = resize(img, 512)
    faces = detect_faces(img)     # heavy CPU / GPU work
    save(img)
    return {"path": path, "faces": len(faces)}

paths = [f"s3://school/photos/{i}.jpg" for i in range(10_000)]

futures = [process_photo.remote(p) for p in paths]   # sends 10,000 tasks to the pods
results = ray.get(futures)                           # waits for all of them
print("done:", len(results))
```



The magic: the Python code is almost normal. Adding `@ray.remote` and `.remote()` is all it takes to spread it across 50 machines.

3. Example 2: train or tune an ML model

You want to test **100 settings** (learning rate, layers and so on) of a "student pass/fail predictor". Each training run takes **1 hour**.

Setup	Time
1 machine, one after another	**100 hours** (4 days)
KubeRay with 20 GPU pods (Ray Tune runs 20 at a time)	**about 5 hours**

```
from ray import tune

def train_model(config):
    model = build(lr=config["lr"], layers=config["layers"])
    acc = model.fit(data)
    tune.report(accuracy=acc)

tune.run(
    train_model,
    config={"lr": tune.loguniform(1e-4, 1e-1), "layers": tune.choice([2, 4, 8])},
    num_samples=100,
    resources_per_trial={"gpu": 1},
)
```

4. Example 3: AI model as an API (RayService)

Serve a **sentiment model** ("is this student feedback positive?"). At 9 AM thousands of requests come in, so KubeRay adds pods. At night it scales down.

```
from ray import serve

@serve.deployment(autoscaling_config={"min_replicas": 1, "max_replicas": 10})
class Sentiment:
    def __init__(self):
        self.model = load_model("sentiment.pt")
    async def __call__(self, request):
        text = (await request.json())["text"]
        return {"positive": self.model.predict(text) > 0.5}

app = Sentiment.bind()
```

5. When to use KubeRay (and when not)

Good fit	Not needed
Image or video processing on thousands of files	Normal CRUD API (NestJS + Postgres)
ML training, tuning, predictions on large data	Sending emails, small background jobs (use **BullMQ**, #57)
Big data cleanup or analysis (like a Python Spark)	Calculations that take less than a few seconds
Simulations, scientific maths, report crunching	When one machine is already fast enough

How it fits with your NestJS app



NestJS takes the request, and KubeRay does the heavy Python work.

Heavy work is in...	On one machine	Across many pods
Python	multiprocessing	**KubeRay**
Node.js	worker_threads / Piscina (#56)	**BullMQ + KEDA** (#57)

PART 3: Local setup (your laptop)

6. The goal of local

Local is for **learning, testing the code and running small data**. It is **not** for speed. We use **100 test photos**, not 10,000.

```
┌──────────────── Your Laptop (Windows) ────────────────┐
│  Docker Desktop                                       │
│    └── Kubernetes (kind / Docker Desktop K8s, 1 node) │
│        ├── KubeRay operator pod                       │
│        ├── Ray head pod     (1 CPU, 2 GB)             │
│        └── Ray worker pods  (1-2 x 1 CPU, 1 GB)       │
│                                                       │
│  Photos: 100 test images in a local folder / MinIO    │
└───────────────────────────────────────────────────────┘
```

7. Local setup steps

```
# 1. a Kubernetes cluster on your laptop
kind create cluster                    # or: Docker Desktop -> Settings -> Kubernetes -> Enable

# 2. install the KubeRay operator with Helm
helm repo add kuberay https://ray-project.github.io/kuberay-helm/
helm repo update
helm install kuberay-operator kuberay/kuberay-operator

# 3. create a small Ray cluster
kubectl apply -f raycluster-local.yaml
kubectl get pods                       # head + 1 worker should be Running

# 4. open the Ray dashboard
kubectl port-forward svc/photo-local-head-svc 8265:8265
#   -> http://localhost:8265

# 5. run the script on the cluster
ray job submit --address http://localhost:8265 --working-dir . -- python process_photos.py --
limit 100
```

raycluster-local.yaml (small sizes)

```
apiVersion: ray.io/v1
kind: RayCluster
metadata: { name: photo-local }
spec:
  headGroupSpec:
    rayStartParams: {}
    template:
      spec:
        containers:
          - name: ray-head
            image: rayproject/ray:2.9.0
            resources: { limits: { cpu: "1", memory: "2Gi" } }
  workerGroupSpecs:
    - groupName: workers
      replicas: 1
      minReplicas: 1
```

```
    maxReplicas: 2            # a laptop can't handle more
    rayStartParams: {}
    template:
      spec:
        containers:
          - name: ray-worker
            image: rayproject/ray:2.9.0
            resources: { limits: { cpu: "1", memory: "1Gi" } }
```

Result: 100 photos x 2 s / 2 workers = **about 100 seconds**. That's enough to see that the code works.

> **Laptop RAM warning:** Docker Desktop + Kubernetes + Ray needs about **4 GB free RAM**. With only about 1-2 GB free, pods crash with `OOMKilled` (out of memory). Close Chrome and other heavy apps first, and keep `maxReplicas` small.

PART 4: Production setup (AWS)

8. The goal of production

Production uses **real data** and gives **real speed**. It must **survive failures**, and you should **pay only while it runs**.

```
┌──────────────────────── AWS ────────────────────────┐
│  EKS (managed Kubernetes)                           │
│   ├── System nodes (2 x t3.medium, always on)       │
│   │     └── KubeRay operator, monitoring            │
│   ├── Ray head node (1 x m5.xlarge: 4 CPU, 16 GB)   │
│   └── Worker nodes  (0 -> 50 x c5.xlarge: 4 CPU, 8 GB) │
│         autoscaled, spot instances (about 70% cheaper) │
│         GPU group (0 -> 5 x g4dn.xlarge) for faces  │
│                                                     │
│  S3:  10,000 photos in, results out                 │
│  RDS Postgres: job status + results                 │
│  ECR: your Docker image with the Python code        │
│  CloudWatch / Grafana: logs, metrics, alerts        │
└─────────────────────────────────────────────────────┘
          ^
  NestJS API (ECS / EKS) ── "start job" ──> creates a RayJob
```

9. Each AWS piece and its job

Service	Job here
EKS	Managed Kubernetes. AWS runs the control plane (#53).
EC2 node groups	The machines. CPU spot nodes for workers, GPU nodes for face detection.
Cluster Autoscaler / Karpenter	Adds EC2 machines when Ray asks for more pods, and removes them afterwards
S3	Input photos and output results (#51)
RDS Postgres	Job status (`queued / running / done / failed`) and summary results
ECR	Stores your Docker image, e.g. `photo-ray:1.4`
IAM roles (IRSA)	Lets pods read S3 without putting keys in the code
CloudWatch / Grafana	Logs, CPU/GPU graphs, and alerts when a job fails

10. Production Dockerfile and RayJob

```
# Dockerfile
FROM rayproject/ray:2.9.0
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt     # opencv, boto3, face model...
COPY process_photos.py .
```

```
docker build -t photo-ray:1.4 .
docker tag photo-ray:1.4 123456.dkr.ecr.ap-south-1.amazonaws.com/photo-ray:1.4
docker push 123456.dkr.ecr.ap-south-1.amazonaws.com/photo-ray:1.4
```

rayjob-prod.yaml

```
apiVersion: ray.io/v1
kind: RayJob
metadata: { name: photo-job-2026-11-01 }
spec:
  entrypoint: python process_photos.py --input s3://school/photos/
  shutdownAfterJobFinishes: true     # cluster deleted afterwards, so you stop paying
  ttlSecondsAfterFinished: 300
  rayClusterSpec:
    enableInTreeAutoscaling: true    # Ray adds or removes workers by itself
```

```
    headGroupSpec:
      rayStartParams: {}
      template:
        spec:
          serviceAccountName: ray-s3-access          # IAM role -> S3 access
          containers:
            - name: ray-head
              image: 123456.dkr.ecr.ap-south-1.amazonaws.com/photo-ray:1.4
              resources: { limits: { cpu: "4", memory: "16Gi" } }
    workerGroupSpecs:
      - groupName: cpu-workers
        minReplicas: 0
        maxReplicas: 50
        rayStartParams: {}
        template:
          spec:
            serviceAccountName: ray-s3-access
            nodeSelector: { node-type: spot-cpu }
            containers:
              - name: ray-worker
                image: 123456.dkr.ecr.ap-south-1.amazonaws.com/photo-ray:1.4
                resources: { limits: { cpu: "4", memory: "8Gi" } }
      - groupName: gpu-workers
        minReplicas: 0
        maxReplicas: 5
        rayStartParams: {}
        template:
          spec:
            nodeSelector: { node-type: gpu }
            containers:
              - name: ray-worker
                image: 123456.dkr.ecr.ap-south-1.amazonaws.com/photo-ray:1.4
                resources: { limits: { cpu: "4", memory: "16Gi", nvidia.com/gpu: "1" } }
```

Result: 10,000 photos / (50 pods x 4 CPU = 200 at once) x 2 s = **about 2 minutes**. Then the cluster is deleted.

11. NestJS starts the job

```
// jobs.service.ts (uses @kubernetes/client-node)
async startPhotoJob(folder: string) {
  const name = `photo-job-${Date.now()}`;
  await this.k8s.createNamespacedCustomObject({
    group: 'ray.io', version: 'v1', namespace: 'ml', plural: 'rayjobs',
    body: buildRayJob(name, `s3://school/${folder}/`),   // the YAML above as an object
  });
  await this.jobRepo.save({ name, status: 'running' });  // Postgres
  return { jobId: name };                                // answer immediately
}
```

PART 5: Local vs production

12. Side-by-side comparison

	Local (laptop)	Production (AWS)
Kubernetes	kind / Docker Desktop (1 node)	EKS (many nodes, managed)
Machines	Your laptop only	1 head + 0-50 workers + GPUs
Pod size	1 CPU, 1-2 GB	4 CPU, 8-16 GB (+ GPU)
Workers	1-2, fixed	0 -> 50, **autoscaled**
Data	100 test photos, local folder / MinIO	10,000+ photos in **S3**
Image	`rayproject/ray` + local code	Your image in **ECR** with a version tag
Results	Printed in the terminal	Saved to **S3 + Postgres**
Time for 10,000 photos	about 2.8 hours (if it doesn't crash)	**about 2 minutes**
Cost	Free	Pay per minute (roughly Rs 100-300 per run with spot)
Failures	You restart it by hand	Ray retries tasks, K8s replaces dead pods
Monitoring	`kubectl logs`, dashboard on localhost:8265	CloudWatch / Grafana + alerts
Security	None needed	IAM roles, private VPC, secrets
Cluster life	Stays until you delete it	RayJob deletes it after the job

13. The full workflow

```
1. Write process_photos.py
        |
2. LOCAL: kind + KubeRay, 100 photos  --> works?  no --> fix code, try again
        | yes
3. docker build --> push to ECR (tag 1.4)
        |
4. PROD: NestJS creates the RayJob
        |
5. KubeRay creates head + workers --> autoscaler adds EC2 machines
        |
6. 10,000 photos processed in about 2 min --> results to S3 / Postgres
        |
7. Job done --> cluster deleted --> machines removed --> billing stops
```

14. Common mistakes

Mistake	What happens	Fix
Big pod sizes on the laptop	Pods stuck `Pending` or `OOMKilled`	1 CPU / 1 GB, `maxReplicas: 2`
Using the `latest` image tag in prod	No idea which code ran	Version tags: `photo-ray:1.4`
A RayCluster left running in prod	50 machines billed all night	Use a **RayJob** with `shutdownAfterJobFinishes`
AWS keys inside the code	Leaked keys	IAM role on the service account (IRSA)
Tiny tasks (1 ms each)	Overhead is bigger than the work	Group them: 1 task = 100 photos
Running heavy work inside NestJS	API freezes	NestJS only **starts** the job, and Ray does the work

15. Summary

- **KubeRay = Ray (Python) on Kubernetes** for heavy work: images, ML training, big data, model APIs.
- `@ray.remote` + `.remote()` spreads work across many pods. 10,000 photos go from **5.5 hours to 2 minutes**.
- **Local:** kind / Docker Desktop, 1-2 small workers, 100 test photos. It's for **checking the code works**.
- **Production:** EKS + spot/GPU nodes + S3 + ECR + RDS + IAM + monitoring. It's for **speed and reliability**.
- **Same code** in both places. Only the **YAML sizes, the data location and the machines** change.
- Use a **RayJob** in production so the cluster is deleted after the job and billing stops.
- For Node.js heavy work, use **Piscina** (one machine) or **BullMQ + KEDA** (many pods) instead.